DSU S25 MODEL ANSWER MSBTE

This Study Material is provided by Campusifyplus.in ©

Made by Team Campusify

THIS IS THE FREE STUDY MATERIAL BY CAMPUSIFY!!

Be Part of Our Community

Telegram Channel - https://t.me/Campusifyy

Whatsapp Channel - https://tinyurl.com/Campusify-channel

Campusify is building the Biggest Diploma/Engineering Student Community in Maharashtra. Be part of the movement!

Join Channel for MSBTE Study Materials



Earn by Referring Campusify Course

Presented by **Campusify** | **10K** Active Members | **2.7L** Students Helped within **1 year**

Refer Our course with your **Third year** friend or cousin and get **10%** commission on each referral.

For More Details : WhatsApp Us at **+91 8291934640**

TY Course Link - https://www.campusifyplus.site

MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION
(Autonomous)
(ISO/IEC - 27001 - 2013 Certified)

Model Answer

SUMMER –25 EXAMINATION

Subject Name: Data Structure Using 'C' Subject Code: **313301**

Important Instructions to examiners:

1) The answers should be examined by key words and not as word-to-word as given in the model answer scheme.
2) The model answer and the answer written by candidate may vary but the examiner may try to assess the understanding level of the candidate.
3) The language errors such as grammatical, spelling errors should not be given more importance (Not applicable for subject English and Communication Skills).
4) While assessing figures, examiner may give credit for principal components indicated in the figure. The figures drawn by candidate and model answer may vary. The examiner may give credit for any equivalent figure drawn.
5) Credits may be given step wise for numerical problems. In some cases, the assumed constant values may vary and there may be some difference in the candidate's answers and model answer.
6) In case of some questions credit may be given by judgement on part of examiner of relevant answer based on candidate's understanding.
7) For programming language papers, credit may be given to any other program based on equivalent concept.

Q. No	Sub Q.N.	Answer	Marking Scheme
1.		**Attempt any five:**	10 Marks
	a) Ans:	**List any four applications of data structure.** **1. Operating Systems:** Managing memory allocation, process scheduling, and file systems using structures like queues, trees, and hash tables. **2. Database Management Systems:** Indexing and organizing data for quick retrieval using B-trees and hash tables. **3. Compilers**: Constructing symbol tables and abstract syntax trees for code analysis. **4. Social Media Platforms:** Representing user connections and content feeds using graphs and linked lists.	
	b) Ans:	**State the LIFo Principle of stack.** The LIFO (Last In, First Out) principle, as applied to a stack data structure, means that the last element added to the stack is the first one to be removed.	

	c) Ans:	**Define heap data structure.** A Heap is a complete binary tree data structure that satisfies the heap property: for every node, the value of its children is greater than or equal to its own value. Heaps are usually used to implement priority queues, where the smallest (or largest) element is always at the root of the tree.	
	d) Ans:	**List any four applications of stack data structure.** **Applications of Stacks:** • **Function calls:** Stacks are used to keep track of the return addresses of function calls, allowing the program to return to the correct location after a function has finished executing. • **Recursion:** Stacks are used to store the local variables and return addresses of recursive function calls, allowing the program to keep track of the current state of the recursion. • **Expression evaluation:** Stacks are used to evaluate expressions in postfix notation (Reverse Polish Notation). • **Syntax parsing:** Stacks are used to check the validity of syntax in programming languages and other formal languages. • **Memory management:** Stacks are used to allocate and manage memory in some operating systems and programming languages.	
	e) Ans:	**Define the term binary tree and binary search tree.** **Binary Tree :** A binary tree is a hierarchical data structure where each node can have at most two children, typically referred to as the left child and the right child. **binary search tree (BST) :** A binary search tree (BST) is a specific type of binary tree that maintains an ordered structure, where for every node, all nodes in its left subtree have values less than the node's value, and all nodes in its right subtree have values greater than the node's value.	
	f) Ans:	**Define searching. State the two methods of searching.**Searching is the process of locating a specific element within a collection of data. The two primary methods of searching are linear search and binary search. 1.Linear Search 2. Binary Search	
	g) Ans:	**List types of Queue data structure.** A Queue is a linear collection of elements that follows the First-In, First-Out (FIFO) principle. There are several types of queues, including: i)Simple/Linear Queue ii)Circular Queue iii)Priority Queue iv)Double-Ended Queue(Dequeue).	

	2	Attempt any three:	12 Marks

a) **Sort the following array in ascending order using insertion sort**
30,10,40,50,20,45

Ans:

Given array = 30, 10, 40, 50, 20, 45

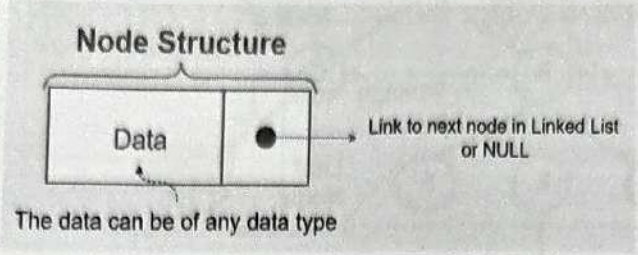
Pass	Elements	Operations.
Pass 1	30, 10, 40, 50, 20, 45	Compare 30 with 10 & insert 10 at index 0.
Pass 2	10, 30, 40, 50, 20, 45	Compare 30 with 40, no change.
Pass 3	10, 30, 40, 50, 20, 45	Compare 40 with 50, no change
Pass 4	10, 30, 40, 50, 20, 45	Compare 40 with 20. and 20 insert at index 1
Pass 5	10, 20, 30, 40, 50, 45	Compare 50 with 45. and insert 45 at index 4
Pass 6	10, 20, 30, 40, 45, 50	Sorted string.

	b)	**Write 'C' program to sort an array of strings using the bubble sort algorithm.**	

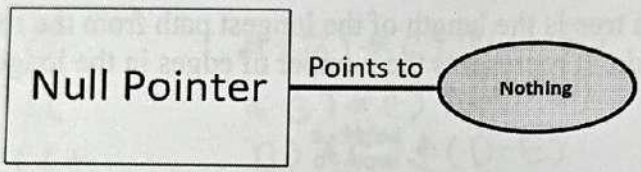
	Ans:	#include<stdio.h>
		#include<conio.h>
		int main(void)
		{
		int a[50],n ,pass,i,j,temp=0;
		
		clrscr();
		printf("enter the value of n:\n");
		scanf("%d",&n);
		printf("enter n element:\n");
		for (i=0;i<n;i++)
		{
		scanf("%d",&a[i]);
		}
		for(pass=1;pass<=n-1;pass++)
		{
		for(j=0;j<n-pass;j++)
		{
		if(a[j]>a[j+1])
		{
		temp=a[j+1];
		a[j+1]=a[j];
		a[j]=temp;
		}
		}
		}
		printf("the sorted array after bubble sort is:\n");
		for(i=0;i<n;i++)
		{
		printf("%d",a[i]);
		}
		getch ();
		}	

	c)	**With neat diagram describe any two terms from the following: Node, Null Pointer, Empty list with respect to singly linked list.**	
	Ans:	**i) Node:** A node in a singly linked list contains two main parts: data and a pointer to the next node. **Data:** This part holds the actual information stored in the node. It can be an integer, a string, or any other data type relevant to the list's purpose. **Next Pointer:** This pointer stores the memory address of the next node in the	

sequence. If it's the last node, this pointer will be NULL



ii)Null Pointer:
In a singly linked list, the "next" pointer of the last node is always set to NULL.
NULL indicates that there are no more nodes in the list.
It's a special value that signifies the end of the list.
It is often represented as 0 or nullptr depending on the programming language.



iii)Empty List:
An empty list in a singly linked list is when there are no nodes present.
The head pointer, which is the entry point to the linked list, will be NULL.
If you try to traverse an empty list, you will immediately encounter NULL.
Operations like insertion and deletion need to handle the empty list case to avoid errors.

d) **Describe following terms with respect to tree data structure**
i)Leaf node of a tree
ii)Sibling
iii)Degree of a tree
iv) Depth of a tree

Ans: **i) Leaf Node:**
A node in a tree is called a leaf node if it does not have any child nodes. In other words, it is a terminal node at the end of a branch.
ii) Sibling:
Nodes that share the same parent are called siblings. For example, if node A has two child nodes, B and C, then B and C are siblings.

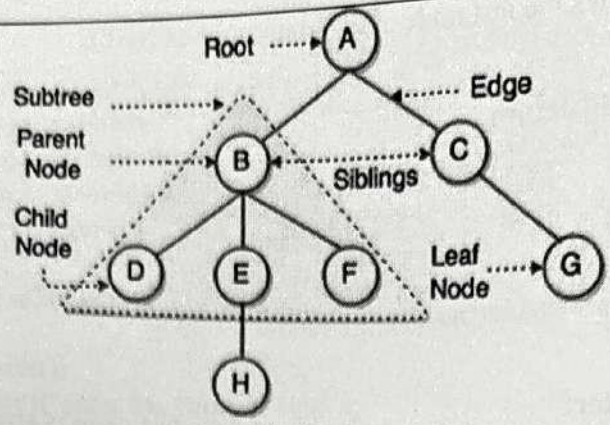
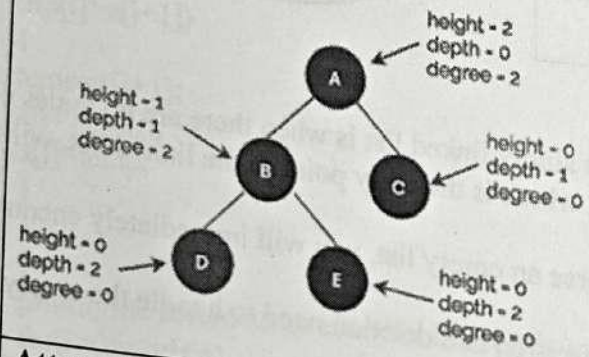


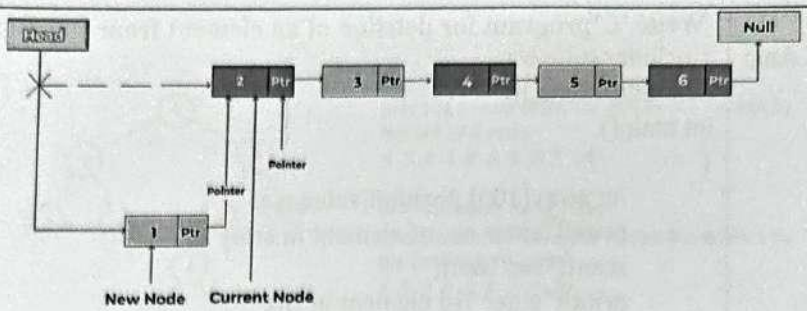
Fig. Structure of Tree

iii) Degree of a Tree:
The degree of a tree is determined by the node with the most children. It's the maximum number of subtrees any node possesses within the tree.

iv)Depth of a Tree:
The depth of a tree is the length of the longest path from the root node to any leaf node. It represents the number of edges in the longest path.



3		**Attempt any three:**	
	a)	**Write an algorithm to insert an element at the beginning of singly linked list.**	**12 Marks**
	Ans:	**Algorithm:** **Step 1:** IF pointer = NULL Then Go to step 7 that is exit otherwise follow step2 [END OF IF] **Step 2:** SET New_Node = pointer **Step 3:** SET pointer = pointer → next **Step 4:** SET New_Node → data= value **Step 5:** SET New_Node →next =head **Step 6:** SET head = New_Node **Step 7:** Exit	



b) **Write down step-by-step conversion of the following infix expression to postfix expression using stack**
((A+B)*C)^(D-E)

Ans:

Stack	Input	output stack
Empty	((A+B)*C)↑(D-E)	-
(	(A+B)*C)↑(D-E)	-
((	A+B)*C)↑(D-E)	-
((+	B)*C)↑(D-E)	A
((+	)*C)↑(D-E)	AB
((+)	*C)↑(D-E)	AB+
(*	C)↑(D-E)	AB+
(*	)↑(D-E)	AB+C
(*)	↑(D-E)	AB+C*
↑	(D-E)	AB+C*
↑(	D-E)	AB+C*
↑(	-E)	AB+C*D
↑(-	E)	AB+C*D
↑(-	)	AB+C*DE
↑(-)	Empty	AB+C*DE-
↑	Empty	AB+C*DE-↑
Empty	Empty	AB+C*DE-↑

c) **Write 'C'program for deletion of an element from an array.**

Ans:

```
#include<stdio.h>
#include<conio.h>
int main()
{
        int array[100],position,value,n,a;
        printf("enter no. of element in array");
        scanf("%d",&n);
        printf("enter %d element \n",n);
        for(a=0;a<n;a++)
        scanf("%d",&array[a]);
        printf("enter location where you wish to delete an element\n");
        scanf("%d",&position);
        if(position>=n-1)
        printf("deletion not possible");
        else
        for(a=position-1;a<n;a++)
        array[a]=array[a+1];
        printf("resultant array is:\n");
        for(a=0;a<n-1;a++)
        {
        printf("%d\n",array[a]);
        }
        return 0;
        getch();
        }
```

d) **Write an algorithm for in order and post order traversal of binary tree.**

Ans:

- **Inorder Traversal:**

Inorder traversal visits the node in the order: Left -> Root -> Right

Algorithm for Inorder Traversal:

Inorder(tree)

- Traverse the left subtree, i.e., call Inorder(left->subtree)
- Visit the root.
- Traverse the right subtree, i.e., call Inorder(right->subtree)

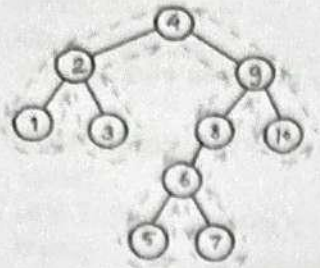
- **Postorder Traversal:**

Postorder traversal visits the node in the order: Left -> Right -> Root

Algorithm for Postorder Traversal:

Postorder(tree)

- Traverse the left subtree, i.e., call Postorder(left->subtree)
- Traverse the right subtree, i.e., call Postorder(right->subtree)
- Visit the root



pre-order node, left, right
process a node when the dotten line passes by the left of a node
4 2 1 3 9 8 6 5 7 10

in-order - left, node, right
process a node when the dotten line passes by the bottom of a node
1 2 3 4 5 6 7 8 10

post-order - left, node, right
process a node when the dotten line passes by the right of a node
1 3 2 5 7 6 8 10 9

4 **Attempt any three:** **12 Marks**

a) **In an array find the position of element 30 using binary search method.**

Ans:

We take array = A = {10, 5, 20, 25, 8, 30, 40}

Mid formula = $\frac{i+j}{2}$

A = {10, 5, 20, 25, 8, 30, 40}

i = 0 1 2 3 4 5 6

Sort array = {5, 8, 10, 20, 25, 30, 40}

i = 0 1 2 3 4 5 6

$\frac{0+6}{2} = \frac{6}{2} = 3$ is middle index of element is 20

30 is greater than 20 so skip the left part and search in Right part

A = {20, 25, 30, 40}

i = 3 4 5 6

$mid = \frac{3+6}{2} = \frac{9}{2} = 4$ index 25 element now search in Right

A = {30, 40}

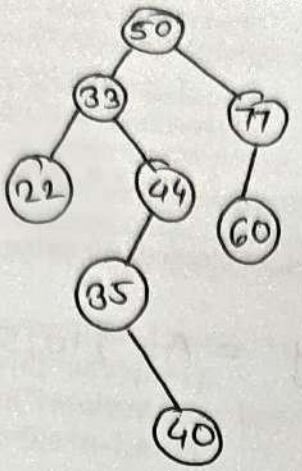
i = 5 6

$mid = \frac{5+6}{2} = \frac{11}{2} = 5$. index element 30

So search element 30 found at index 5.

b) **Construct a binary search tree for the given numbers:**
50,33,44, 22,77,35,60,40
Present each step of construction of BST diagrammatically.

Ans:



Solve step By step.

c) **Write a C program to implement singly linked list with operation**
i)insert at end
ii)Display the list

Ans:

```
#include<stdio.h>
#include<malloc.h>
#include<process.h>
void create();
void insert_at_end();
void display();
struct node
{
 int info;
 struct node *link;
 } *start=NULL;
int data,item,n1,pos,i,m;
void main()
{
 int n;
 setbuf(stdout,NULL);
 printf("\n****linked list****");
 printf("\nn1.create\nn2.insert at end \nn3.display\nn4.exit\n");
 while(1)
```

```
    {
     printf("\nenter your choice:(1.create 2.insert at end 3.insert after
4.display 5.exit)\n");
     scanf("%d",&n);
     switch(n)
     {
      case 1:create();
      break;
      case 2: insert_at_end();
      break;
      case 3:display();
      break;
      exit(0);
      default:
      printf("\n wrong choice!!\n");
      }
     }
    getch();
   }
   void create()
   {
    struct node *q, *tmp;
    printf("enter element:\n");
    scanf("%d",&data);
    tmp=malloc(sizeof(struct node));
    tmp->info=data;
    tmp->link=NULL;
    if(start==NULL)
     start=tmp;
    else
    {
    q=start;
    while(q->link!=NULL)
     q=q->link;
     q->link=tmp;
     }
    }

   void insert_at_end()
   {
    struct node *tmp, *q;
     printf("enter the element to be inserted\n");
     scanf("%d",&data);
     tmp=malloc(sizeof(struct node));
     tmp->info=data;
     tmp->link=NULL;
```

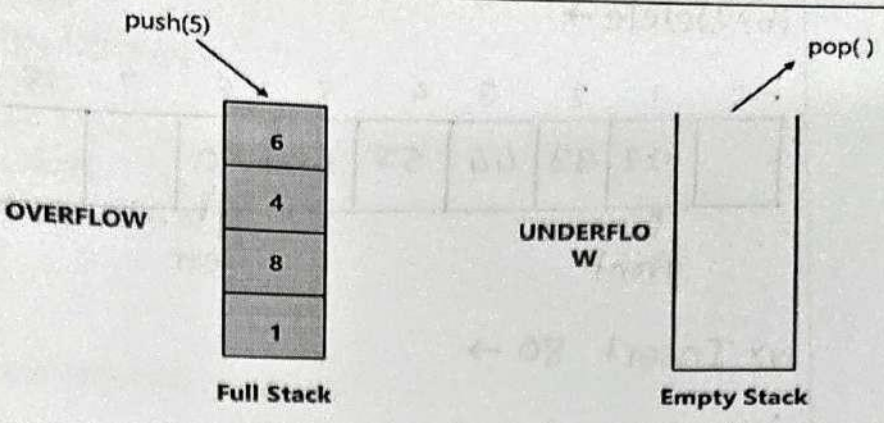
```
if(start==NULL)
     start=tmp;
else
{
    q=start;
    while(q->link!=NULL)
     q=q->link;
     q->link=tmp;
     }
     display();
}
      void display()
      {
      struct node *q;
      if(start==NULL)
      printf("list is empty!!\n");
      else
      {
       printf("*****element in linked list***\n");
       q=start;
       while(q!=NULL)
       {
        printf("%d\t",q->info);
        q=q->link;
        }
       }
      }
```

d) **Describe stack over flow and stack underflow condition with the help of example**

Ans: A stack overflow occurs when a program tries to store more data on the stack than the allocated memory can hold, while a stack underflow occurs when a program tries to retrieve data from an empty stack. Imagine a stack of plates: pushing too many plates onto the stack without removing any is like a stack overflow; trying to take a plate from an empty stack is a stack underflow.

Stack Overflow:
Stack overflow happens when a program recursively calls itself, or pushes excessive data onto the stack, exceeding the stack's capacity.

Stack Underflow:
Stack underflow occurs when a program tries to pop (remove) an element from an empty stack.

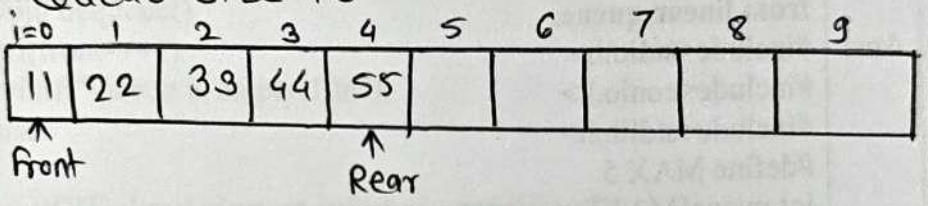


e) **Show the efect of insert and delete operations on to the linear queue o size 10. The linear queue sequentially contain 11, 22,33,44 and 55 where 11 is at front of queue**
Show diagrammatically the effect of
i) INSERT (60)
ii) INSERT(70)
iii) Delete
iv) INSERT(80)

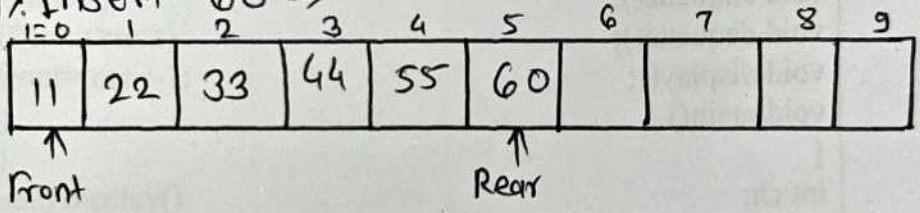
Ans:

Given Queue- 11, 22, 33, 44, 55,

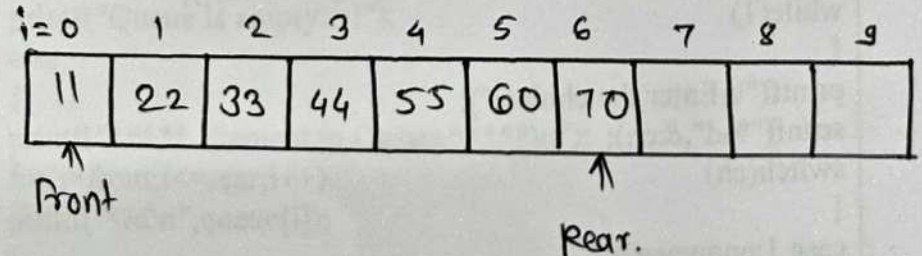
Queue size 10



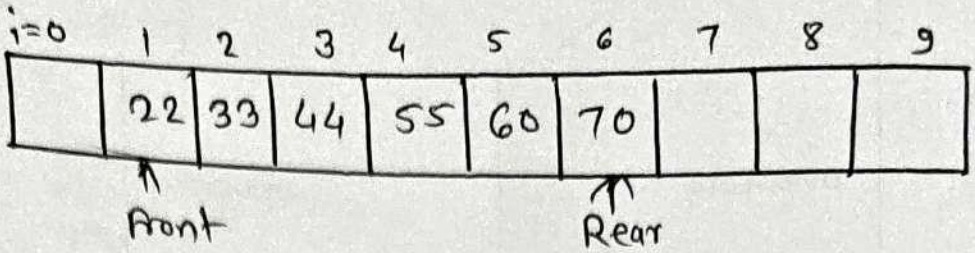
i) Insert 60 →



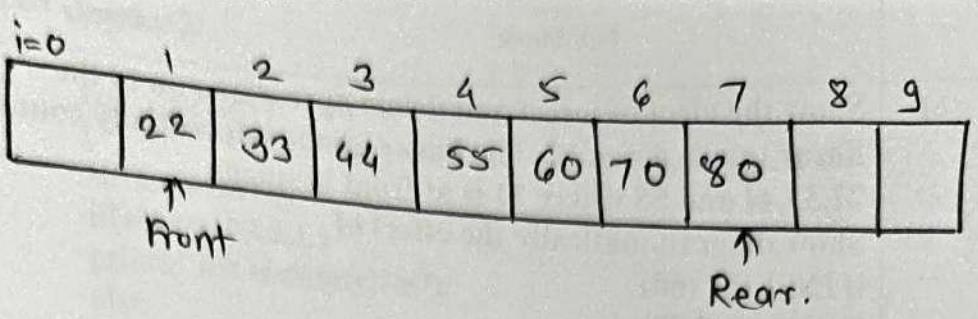
ii) Insert 70 →



iii) Delete →



iv) Insert 80 →



5 **Attempt Any Two :**

a) **Write'C'program to insert into and delete the element into from linear queue.** **12 Marks**

Ans:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#define MAX 5
int queue[MAX],rear=-1,front=-1;
void enqueue();
void dequeue();
void display();
void main()
{
int ch;
clrscr();
printf("**** Queue Menu *****");
printf("\n1.Insert\n2.Delete\n3.Display\n4.Exit");
while(1)
{
printf("\nEnter the choice:");
scanf("%d",&ch);
switch(ch)
{
case 1:enqueue();
break;
case 2:dequeue();
```

```
break;
case 3:display();
break;
case 4:exit(0);
default:
printf("\nWrong choice");
}
}
}
void enqueue()
{
if(rear==MAX-1)
printf("\n Queue is full!!!");
else
{
if(front==-1)
front=0;
rear++;
printf("\nEnter element to be insert:");
scanf("%d",&queue[rear]);
}
}
void dequeue()
{if(front==-1)
printf("Queue is empty!!!\n");
else
{
printf("Deleted element is:%d\n",queue[front]);
front++;
if(front>rear)
front=rear=-1;
}
}
void display()
{
int i;
if(rear==-1)
printf("Queue is empty!!!!");
else
{
printf("***** Element in Queue *****\n");
for(i=front;i<=rear;i++)
printf("%d\n",queue[i]);
}
}
```

b) **Differentiate between singly linked list and doubly linked list (minimum six points)**

Ans:

Difference between Singly Linked List and Doubly Linked List

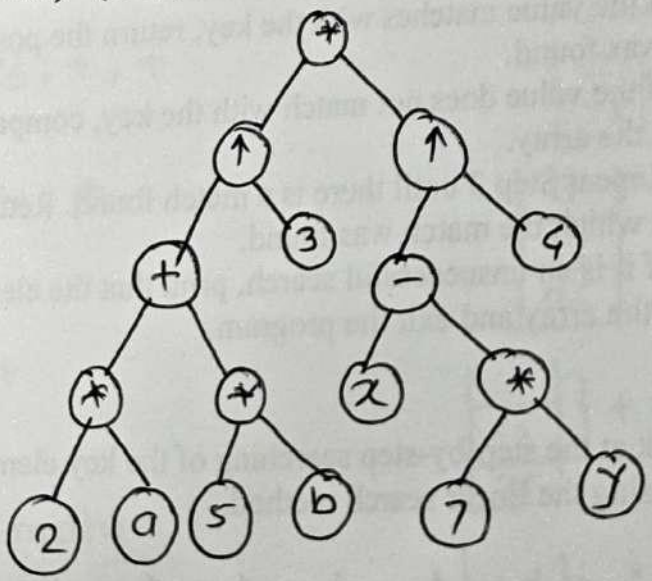
S. No.	Singly linked list	Doubly linked list
1	In case of singly linked lists, the complexity of insertion and deletion is O(n)	In case of doubly linked lists, the complexity of insertion and deletion is O(1)
2	The Singly linked list has two segments: data and link.	The doubly linked list has three segments. First is data and second, third are the pointers.
3	It permits traversal components only in one way.	It permits two way traversal.
4	We mostly prefer a singly linked list for the execution of stacks.	We can use a doubly linked list to execute binary trees, heaps and stacks.
5	When we want to save memory and do not need to perform searching, we prefer a singly linked list.	In case of better implementation, while searching, we prefer a doubly linked list.
6	A singly linked list consumes less memory as compared to the doubly linked list.	The doubly linked list consumes more memory as compared to the singly linked list.

c) **Draw the expression tree of the following expressions.**

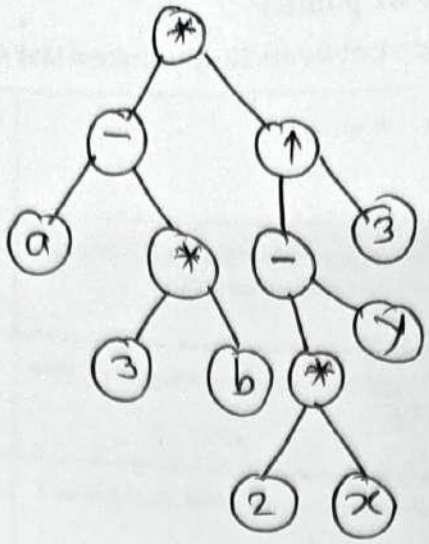
i) $(2a+5b)^3*(x-7y)^4$

ii) $(a-3b)*(2x-y)^3$

Ans: **i)** $(2a+5b)^3*(x-7y)^4$

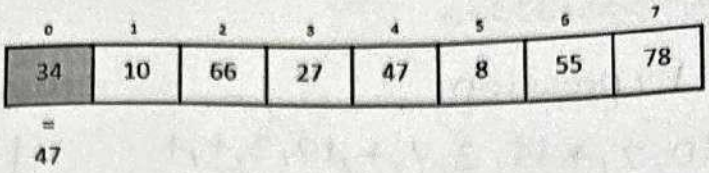


ii)(a-3b)*(2x-y)^3



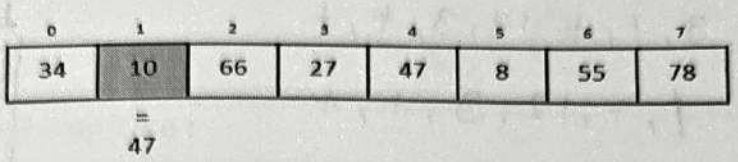
6.		Attempt Any Two :	12 Marks
	a) Ans:	**Describe the working of linear search with example.** Linear search, also known as sequential search, works by examining each element in a list or array one by one until a match is found or the end of the list is reached. If a match is found, the index of that element is returned. If the search reaches the end of the list without finding a match, it indicates that the target element is not present in the list. **Linear Search Algorithm:** The algorithm for linear search is relatively simple. The procedure starts at the very first index of the input array to be searched. **Step 1** – Start from the 0th index of the input array, compare the key value with the value present in the 0th index. **Step 2** – If the value matches with the key, return the position at which the value was found. **Step 3** – If the value does not match with the key, compare the next element in the array. **Step 4** – Repeat Step 3 until there is a match found. Return the position at which the match was found. **Step 5** – If it is an unsuccessful search, print that the element is not present in the array and exit the program. **Example:** Let us look at the step-by-step searching of the key element (say 47) in an array using the linear search method. 0 1 2 3 4 5 6 7 34 10 66 27 47 8 55 78 **Step 1:** The linear search starts from the 0th index. Compare the key	

element with the value in the 0th index, 34.



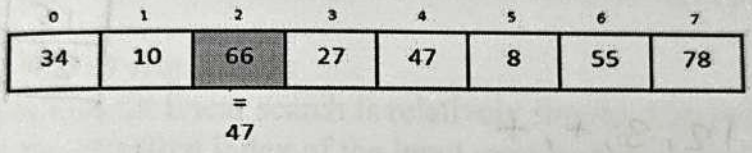
However, 47 34. So it moves to the next element.

Step 2: Now, the key is compared with value in the 1st index of the array.

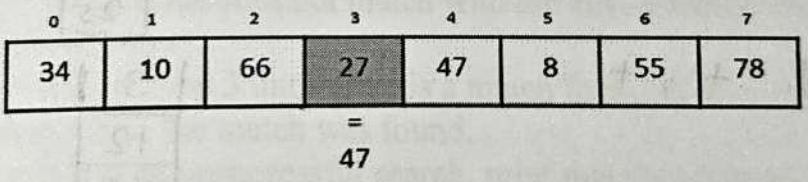


Still, 47 10, making the algorithm move for another iteration.

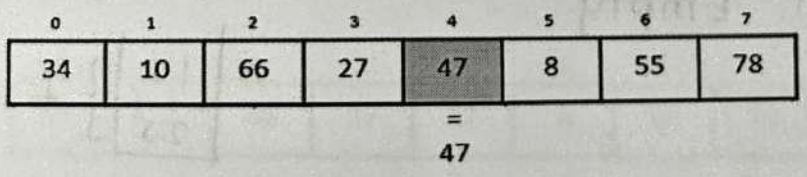
Step 3: The next element 66 is compared with 47. They are both not a match so the algorithm compares the further elements.



Step 4: Now the element in 3rd index, 27, is compared with the key value, 47. They are not equal so the algorithm is pushed forward to check the next element.



Step 5: Comparing the element in the 4th index of the array, 47, to the key 47. It is figured that both the elements match. Now, the position in which 47 is present, i.e., 4 is returned.



The output achieved is Element found at 4th index.

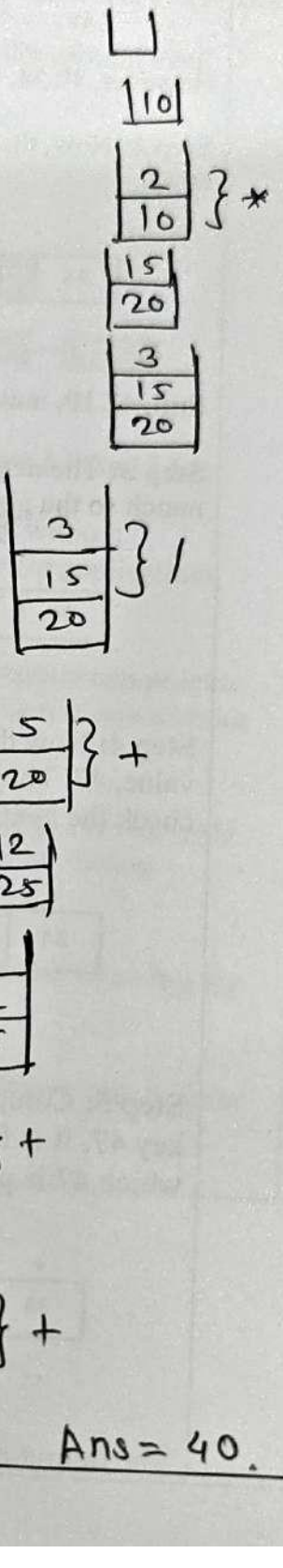
b) **Evaluate the following post fix expression with suitablediagram using stack**

Ans: **10,2,*15,3,/,+,12,3,+,+**

Expression

1. 10, 2, * 15, 3, /, +, 12, 3, +, +
2. 2, *, 15, 3, /, +, 12, 3, +, +
3. 15, 3, /, +, 12, 3, +, +
4. 3, /, +, 12, 3, +, +
5. /, +, 12, 3, +, +
6. +, 12, 3, +, +
7. 12, 3, +, +
8. 3, +, +
9. +, +
10. +
11. Empty
12. Empty

Stack



c) **Describe circular linked list with suitable diagram.Also state the advantages of circular linked list over linear linked list.**

Ans: - A circular linked list is a special type of linked list where all the nodes are connected to form a circle. Unlike a regular linked list, which ends with a node pointing to NULL, the last node in a circular linked list points back to the first node.

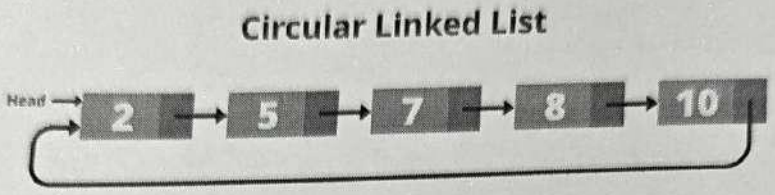
-This means that you can keep traversing the list without ever reaching a NULL value.

-The basic operations on a circular linked list are:

i) Insertion

ii)Deletion and

iii)Traversal



Advantages of Circular Linked List over Linear Linked List:

1. Efficient Traversal:
Circular linked lists allow for continuous traversal without encountering a null pointer, making them suitable for applications requiring cyclic processing.

2. Queue Implementation:
They simplify queue implementation, as the rear pointer can point to the front, enabling efficient enqueue and dequeue operations without traversing the entire list.

3. Flexibility:
Circular linked lists can be traversed from any node, as the last node points back to the first, providing more flexibility in accessing elements.

4. Dynamic Size:
Like linear linked lists, circular linked lists are dynamic, allowing for efficient addition or removal of nodes as needed.

5. No Null at the End:
Unlike linear linked lists, circular linked lists don't have a null pointer at the end, which can simplify certain operations.